

Cricket Analytics SaaS

T20 Match Outcome Prediction Platform

Complete Step-by-Step Implementation Blueprint | 8 Phases | 20-24 Weeks

This document provides a comprehensive, phase-by-phase execution plan for building a production-grade, multi-tenant SaaS platform for T20 cricket match outcome prediction. The architecture covers data ingestion from Cricsheet via Apache Kafka, a Medallion Data Warehouse with dbt and PostgreSQL, XGBoost-based win-probability ML models with SHAP interpretability, an Agentic RAG system using LangGraph and Claude, three persona-specific Next.js dashboards, and a fully containerized Kubernetes deployment on AWS with Stripe monetization. Each phase lists concrete sub-steps, the exact tech stack, and a testable deliverable.

Table of Contents

Phase 1 -- Phase 1: Project Setup & Environment	Week 1-2
Phase 2 -- Phase 2: Data Ingestion & Kafka Streaming	Week 2-4
Phase 3 -- Phase 3: Medallion Data Warehouse (Silver to Gold)	Week 4-7
Phase 4 -- Phase 4: Backend API (FastAPI)	Week 7-9
Phase 5 -- Phase 5: ML Pipeline (XGBoost + Feature Engineering)	Week 9-13
Phase 6 -- Phase 6: GenAI -- Agentic RAG + Text-to-SQL	Week 13-16
Phase 7 -- Phase 7: Frontend Dashboards (Next.js)	Week 16-20
Phase 8 -- Phase 8: Deployment, Security & Monetization	Week 20-24

Bootstrap your dev environment, repo structure, and all tooling before writing a single line of application code.

1.1 Initialize monorepo structure

- Create a Git monorepo with three root directories: /backend, /frontend, /infra.
- Initialize Python 3.11+ virtual environment inside /backend using uv or poetry.
- Initialize Node.js 20+ project inside /frontend with pnpm.
- Add a root .gitignore covering .env files, __pycache__, node_modules, *.pyc, .DS_Store.
- Set up pre-commit hooks: black + isort for Python, prettier + eslint for JS.

STACK: Git Python 3.11 Node 20 uv/poetry pnpm pre-commit

DELIVERABLE *Clean repo with enforced code style from day one.*

1.2 Docker Compose local stack

- Write docker-compose.yml in /infra defining all local services: postgres, redis, kafka, zookeeper, kafka-ui.
- Use named volumes for Postgres and Redis data so containers restart without data loss.
- Expose Kafka on 9092, Postgres on 5432, Redis on 6379, Kafka-UI on 8080.
- Add a Makefile at root with targets: make up, make down, make logs, make shell.
- Test: run make up and verify all containers are healthy via docker ps.

STACK: Docker Docker Compose Kafka Zookeeper PostgreSQL Redis

DELIVERABLE *One-command local environment via: make up.*

1.3 Environment & secrets management

- Create .env.example files in /backend and /frontend listing all required vars (never commit .env).
- Required vars: DATABASE_URL, REDIS_URL, KAFKA_BOOTSTRAP_SERVERS, ANTHROPIC_API_KEY, SECRET_KEY, JWT_SECRET.
- Install python-dotenv in backend; use Next.js built-in env handling in frontend.
- For production plan to use AWS Secrets Manager or Doppler -- document this in README now.
- Add GitHub Actions workflow skeleton: .github/workflows/ci.yml with placeholder test step.

STACK: dotenv GitHub Actions AWS Secrets Manager

DELIVERABLE *.env.example files + CI pipeline skeleton ready.*

Build the data acquisition layer -- download Cricsheet data, parse it, and stream it through Kafka to simulate real-time ball-by-ball feeds.

2.1 Download & parse Cricsheet data

- Write scripts/download_cricsheet.py: fetch the Cricsheet bulk ZIP, extract all JSONs into /data/raw/.
- Parse Cricsheet JSON: extract meta (version), info (venue, teams, toss, players), innings (ball-by-ball array).
- Write CricsheetParser class in backend/ingestion/parser.py flattening nested delivery objects into row dicts: {match_id, inning, over, ball, batter, bowler, runs_off_bat, extras, is_wicket, dismissal_kind}.
- Handle null semantics: extras=null to 0, wicket=null to False. Validate: max 10 wickets, max 20 overs.
- Unit test the parser with 5 sample match JSONs: super overs, no-balls, retired hurt edge cases.

STACK: `Python` `requests` `json` `dataclasses` `pytest`

DELIVERABLE *CricsheetParser class with all unit tests passing.*

2.2 Build Kafka producer (live simulator)

- Create backend/ingestion/producer.py using kafka-python or confluent-kafka library.
- Implement generator pattern: yield one delivery dict at a time -- never load all matches into memory.
- Add configurable DELIVERY_INTERVAL_MS env var (default 500ms). Testing uses 50ms; live mode uses 45000ms.
- Inject event_timestamp field (datetime.utcnow().isoformat()) to each message before publishing.
- Publish to Kafka topic 'cricket.deliveries' using match_id as partition key.
- Add second topic 'cricket.match_events' for match-level events: match_start, innings_break, match_end.

STACK: `kafka-python` `confluent-kafka` `Python generators`

DELIVERABLE *Producer script streaming 1 delivery per 500ms to Kafka topic.*

2.3 Build Kafka consumer & Bronze layer sink

- Create backend/ingestion/consumer.py: subscribe to 'cricket.deliveries', commit offsets manually for at-least-once guarantees.
- Sink raw JSON payloads to /data/bronze/ partitioned by date: bronze/year=2024/month=01/day=15/batch_001.json.
- Write to bronze_deliveries table in PostgreSQL: columns raw_payload JSONB, kafka_offset BIGINT, ingested_at TIMESTAMP.
- Implement dead-letter queue: failed messages go to 'cricket.deliveries.dlq' with error metadata.
- Verify: bronze table row count must match producer message count exactly.

STACK: `kafka-python` `PostgreSQL` `JSONB` `psycopg2`

DELIVERABLE *Full pipeline: producer to Kafka to consumer to Bronze PostgreSQL table.*

Transform raw Bronze JSONB into clean Silver tables, then build the Star Schema Gold layer with fact and dimension tables powering all analytics and ML.

3.1 Silver layer: clean & flatten deliveries

- Install dbt-postgres. Run dbt init cricket_analytics inside /backend/dbt/.
- Create models/silver/stg_deliveries.sql: SELECT from bronze_deliveries, use jsonb_extract_path_text() to pull each field, CAST to correct types.
- Transformations: COALESCE(extras, 0), CAST(is_wicket AS BOOLEAN), UPPER(dismissal_kind), standardize player names via seed CSV mapping Cricsheet IDs to global IDs.
- Add dbt tests in schema.yml: not_null on match_id/batter/bowler, accepted_values for dismissal_kind.
- Create stg_match_info.sql: one row per match with venue, toss_winner, toss_decision, match_date, teams.

STACK: dbt PostgreSQL SQL JSONB

DELIVERABLE Silver layer: clean stg_deliveries and stg_match_info tables.

3.2 Gold layer: dimension tables

- Create dim_player.sql: DISTINCT player data, surrogate key via dbt_utils.generate_surrogate_key(), batting_style, bowling_style, national_team. Implement SCD Type 2 via dbt snapshots for role changes.
- Create dim_venue.sql: stadium_name, city, country, pitch_type (Spin/Pace/Flat/Batting), average_first_innings_score, boundary_size_category.
- Create dim_match_context.sql: tournament_phase (Group/Super8/Semifinal/Final), toss_winner_key, toss_decision, weather_category, day_night_flag.
- Create dim_date.sql: date spine via dbt_utils.date_spine(), day_of_week, is_weekend, tournament_week_number.
- Seed dim_venue_manual.csv with curated pitch data for all 2026 World Cup venues.

STACK: dbt dbt_utils SCD Type 2 dbt snapshots

DELIVERABLE 4 dimension tables in Gold layer with surrogate keys.

3.3 Gold layer: fact tables

- Create fact_delivery.sql: one row per ball with FK columns to all dimensions, runs_off_bat, extras, is_wicket, boundary_flag, is_dot_ball, win_probability_delta (NULL initially), tenant_id.
- Create fact_match_summary.sql: aggregated per match -- total_runs_team_a/b, wickets, winner_team_key, margin_runs, margin_wickets, toss_winner_won BOOL.
- Cross-check: SUM(runs_off_bat) in fact_delivery must equal total_runs in fact_match_summary per match.
- Add elementary package for automated data quality monitoring and observability dashboard.

STACK: dbt PostgreSQL elementary dbt tests

DELIVERABLE Complete Star Schema: 2 fact tables + 4 dimension tables with referential integrity.

3.4 Multi-tenancy: Row-Level Security

- Add tenant_id VARCHAR column to all Gold layer tables (fact_delivery, fact_match_summary).
- PostgreSQL policy: CREATE POLICY tenant_isolation ON fact_delivery USING (tenant_id = current_setting('app.current_tenant')).
- Enable RLS: ALTER TABLE fact_delivery ENABLE ROW LEVEL SECURITY.
- FastAPI: execute SET app.current_tenant = '{id}' on every DB connection before any query.
- Integration test: verify tenant A's queries return 0 rows from tenant B's data.

STACK:

PostgreSQL RLS

Row-Level Security

FastAPI

DELIVERABLE *Tenant-isolated data warehouse with zero cross-tenant data leakage.*

Build the FastAPI backend -- authentication, tenant management, data access, and API endpoints for all three user personas.

4.1 FastAPI project structure & core setup

- Structure: backend/app/main.py, /routers/, /models/, /schemas/, /dependencies/, /services/, /db/.
- Install: fastapi, uvicorn, sqlalchemy 2.0, alembic, pydantic v2, python-jose, passlib, httpx.
- Configure SQLAlchemy 2.0 async engine with asyncpg driver pointing to PostgreSQL.
- Run alembic init for DB migrations. Create initial migration: users, tenants, subscriptions tables.
- Add CORS middleware allowing the Next.js frontend origin, and GZipMiddleware for compression.

STACK: FastAPI SQLAlchemy 2.0 Alembic asyncpg Pydantic v2

DELIVERABLE FastAPI app boots, DB migrations run, /health endpoint returns 200.

4.2 Authentication & tenant context injection

- JWT auth: POST /auth/register and POST /auth/login return access_token (15min) + refresh_token (7 days).
- get_current_user FastAPI dependency: decodes JWT, fetches user from DB, returns User object.
- get_tenant_context dependency: extracts tenant_id and injects via SET app.current_tenant into every DB session.
- Role-based access control: roles [fan, professional, enterprise, admin]. Decorate endpoints with @require_role().
- Rate limiting via slowapi: Fan = 100 req/hour, Professional = 1000/hour, Enterprise = unlimited.

STACK: JWT python-jose passlib slowapi

DELIVERABLE Auth endpoints working, tenant isolation enforced on every DB query.

4.3 Core analytics API endpoints

- GET /matches: paginated list of matches with filters (team, venue, date_range, tournament_phase).
- GET /matches/{match_id}/win-probability: fetch from Redis cache first, fallback to ML inference.
- GET /players/{player_id}/stats: rolling averages (last 10 innings), phase-wise strike rates, head-to-head matchup stats.
- GET /venues/{venue_id}/analytics: average scores, toss impact, pitch type behavior, chase success rate.
- GET /tournaments/{id}/leaderboard: team ELO rankings, top batters/bowlers by phase.
- All endpoints return Pydantic response models with type hints. OpenAPI docs at /docs.

STACK: FastAPI routers Redis Pydantic v2 OpenAPI

DELIVERABLE 6 core endpoints documented at /docs returning correct data.

Build the feature engineering pipeline, train the XGBoost win-probability model, integrate SHAP for interpretability, and deploy via MLflow + Ray Serve.

5.1 Feature engineering pipeline

- Create backend/ml/features.py with FeatureEngineer class. Input: all deliveries up to ball N. Output: 25-feature vector.
- Dynamic match state features: balls_remaining, wickets_in_hand, current_run_rate, required_run_rate, run_rate_differential.
- Player strength features: batter_rolling_sr_last10, bowler_rolling_economy_last10, team_elo_score (sum of playing XI ELO ratings).
- Contextual features: venue_avg_first_innings, toss_venue_impact interaction, pitch_phase (NEW_BALL/POWERPLAY_SPIN/DEATH), is_knockout BOOL.
- Unit tests: given mock match state, assert feature vector correct shape and zero NaN values.

STACK: Python pandas numpy scikit-learn

DELIVERABLE FeatureEngineer class producing a 25-feature vector per delivery.

5.2 Train & evaluate XGBoost model

- Load fact_delivery from PostgreSQL. Target: match_winner (binary 0/1 for team batting first).
- Time-based split: matches before 2023 = train, 2023 = val, 2024 = test. Never shuffle -- no future data leakage.
- Train xgboost.XGBClassifier, optimize log-loss. Use Optuna for hyperparameter search: max_depth, learning_rate, n_estimators, subsample, colsample_bytree.
- Evaluate: log-loss on test set, calibration curve, Brier score.
- Track all experiments in MLflow: log params, metrics, model artifact. Inspect via mlflow ui.

STACK: XGBoost Optuna MLflow scikit-learn pandas

DELIVERABLE Trained model in MLflow with log-loss below 0.45 on test set.

5.3 SHAP interpretability & real-time inference

- Add shap.TreeExplainer to inference pipeline. Return top 3 contributing features with impact magnitude and direction per prediction.
- Build backend/ml/inference.py: load MLflow model, accept match_state dict, run FeatureEngineer, return prob + SHAP explanation.
- Integrate with Redis: after each Kafka delivery event, run inference, cache in Redis key 'wp:{match_id}' with 120s TTL.
- Deploy as Ray Serve endpoint on localhost:8000/predict -- FastAPI calls this internal service.

STACK: SHAP Ray Serve MLflow Redis XGBoost

DELIVERABLE Win probability API returning prob + SHAP explanation in under 200ms.

5.4 Additional ML use cases

- Score forecasting (regression): XGBoostRegressor trained on Powerplay state to predict final score.
- Player clustering (K-Means, K=5): archetypes -- Anchor, Finisher, Powerplay Specialist, Death Bowler, All-Rounder. Store in dim_player.
- Matchup association rules (Apriori via mlxtend): mine frequent {bowler_style to batter_dismissal} patterns. Expose via GET /matchups/vulnerabilities.
- Fantasy points projection: weighted formula = (expected_runs x 1) + (expected_wickets x 25) + (catch_prob x 8).

STACK: XGBoost Regressor scikit-learn KMeans mlxtend Apriori

DELIVERABLE 4 ML models deployed: classifier, regressor, clustering, association rules.

Integrate the Agentic RAG system using LangGraph -- routing natural language queries to Text-to-SQL for structured data or vector search for unstructured commentary.

6.1 Text-to-SQL agent tool

- Install: langchain, langgraph, langchain-anthropic, anthropic.
- Build a schema context string: detailed description of all Gold layer tables, columns, data types, and FK relationships -- injected into LLM system prompt.
- Create tools/text_to_sql.py: LangChain tool that accepts NL query, calls Claude with schema context, receives SQL, sanitizes (no DROP/DELETE/UPDATE), executes in READ-ONLY transaction, returns JSON.
- Add tenant_id injection: programmatically append AND tenant_id = '{id}' to every WHERE clause.
- Test: 'What is the average first innings score at Wankhede?' should produce correct SQL and return the number.

STACK: LangChain LangGraph Claude API SQLAlchemy

DELIVERABLE *Text-to-SQL tool: NL query to correct SQL to numerical answer.*

6.2 Vector search tool for unstructured data

- Collect unstructured documents: pitch curator reports, umpire match reports, expert commentary. Store as .txt in /data/documents/.
- Install: chromadb, langchain-community.
- Chunk documents: split by paragraph, max 500 tokens. Generate embeddings using claude-3-haiku. Store in ChromaDB collection 'cricket_docs'.
- Create tools/vector_search.py: accepts query, embeds it, runs ChromaDB similarity search (top-5), returns retrieved context.

STACK: ChromaDB LangChain Anthropic Embeddings Claude Haiku

DELIVERABLE *Vector store populated, similarity search returning relevant document chunks.*

6.3 LangGraph agentic router

- Build LangGraph StateGraph with nodes: router_node, text_to_sql_node, vector_search_node, synthesizer_node.
- Router node: Claude analyzes query -- if specific numbers/records to text_to_sql; if qualitative/reports to vector_search; if both, invoke in parallel.
- Synthesizer node: combines tool outputs, passes to Claude for final NL answer. System prompt: 'Never hallucinate statistics.'
- Expose via POST /ai/chat in FastAPI. Accept {query, conversation_history}. Return {answer, sources, tool_used}.
- Add conversation memory: store last 5 turns in Redis keyed by session_id for follow-up questions.

STACK: LangGraph LangChain Redis FastAPI

DELIVERABLE *Conversational AI endpoint routing queries to correct tool with grounded answers.*

6.4 Dynamic narrative generation

- Create NarrativeService in backend/services/narrative.py.
- Subscribe to Redis pub/sub 'match.probability.updated'. On win_probability_delta > 0.05, trigger narrative generation.
- Prompt: 'Given this match state and probability change, write 2-sentence broadcast-style commentary for {audience_type}.' Types: coach (tactical), bettor (financial), broadcaster (dramatic).
- Cache narratives in Redis: 'narrative:{match_id}:{over}' with 300s TTL.
- Expose via GET /matches/{id}/narrative?persona=broadcaster.

STACK: Claude API Redis Pub/Sub LangChain

DELIVERABLE *Auto-generated momentum narratives triggered by significant probability swings.*

Build three persona-specific dashboards (Coach, Bettor, Broadcaster) plus the AI chat interface using Next.js with React and Recharts.

7.1 Next.js app setup & shared components

- Initialize: `npx create-next-app@latest` with App Router, TypeScript, Tailwind CSS.
- Install: `recharts`, `@tanstack/react-query`, `axios`, `zustand`, `socket.io-client`, `shadcn/ui`.
- Shared components: `Navbar`, `StatCard`, `WinProbabilityGauge` (semi-circle gauge with live WebSocket updates), `PersonaBadge`.
- React Query for all API calls with 30s stale time. Axios interceptor attaches JWT Bearer token to every request.
- AuthContext with Zustand: stores `user`, `tenant_id`, `subscription_tier`. Login/logout flows end-to-end.

STACK: `Next.js 14` `TypeScript` `Tailwind` `React Query` `Zustand` `shadcn/ui`

DELIVERABLE *Shared component library + auth flow working end-to-end.*

7.2 Coach dashboard

- Route: `/dashboard/coach`. Gate with `@require_role(['professional', 'enterprise'])`.
- Spray chart: Recharts `ScatterChart` showing where a batter hits the ball, colored by shot type.
- Phase-wise strike rate chart: BarChart with 3 bars per player -- Powerplay SR, Middle Overs SR, Death SR.
- Bowler matchup heatmap: batters as rows, bowler styles as columns, dismissal rate per cell. Color: red (vulnerable) to green (dominant).
- Playing XI optimizer: input opponent team, get optimal batting order recommendation via `POST /ai/optimize-xi`.

STACK: `Recharts` `Next.js` `TanStack Query`

DELIVERABLE *Coach dashboard with spray chart, phase analysis, and matchup heatmap.*

7.3 Bettor dashboard & broadcaster view

- Bettor (`/dashboard/bettor`): Live `WinProbabilityGauge` via WebSocket every ball. Historical odds trendline (`LineChart`, last 20 overs). Fantasy Dream XI recommendations with projected points.
- Broadcaster (`/dashboard/broadcaster`): Big live win probability with dramatic color animation on swings. Momentum shift detector timeline with annotated turning points. Real-time milestone alerts.
- WebSocket: connect to `/ws/match/{match_id}`. On each delivery receive `{win_prob, narrative, last_delivery_summary}` and update all reactive components.
- Subdomain routing in Next.js middleware: `teama.app.com` injects `tenant_id='teama'` into all API requests.

STACK: `WebSocket` `Recharts` `Next.js Middleware`

DELIVERABLE *All 3 persona dashboards live with real-time win probability updates.*

7.4 AI chat interface

- Build /dashboard/ai-chat: chat UI with message history, typing indicator, and source citation display.
- Each AI response shows: answer text, 'Tool used' badge (Text-to-SQL or Vector Search), clickable source links.
- Add quick-action prompt chips: 'Best chaser at this venue', 'Compare teams head-to-head', 'Predicted score from Powerplay'.
- Streaming responses: fetch with ReadableStream displays Claude's answer word-by-word (FastAPI Server-Sent Events).
- Gate full AI chat behind Professional/Enterprise. Fan tier gets 5 queries/day with paywall modal.

STACK:

Server-Sent Events

ReadableStream

React

DELIVERABLE *Streaming AI chat interface with tool attribution and tier gating.*

Containerize all services, deploy to cloud with Kubernetes, implement Stripe payments, and harden security for production.

8.1 Dockerize all services

- FastAPI Dockerfile: python:3.11-slim base, copy requirements, run as non-root user (USER 1000), EXPOSE 8000.
- Next.js Dockerfile: node:20-alpine, pnpm install --frozen-lockfile, next build, EXPOSE 3000.
- ML inference Dockerfile (Ray Serve): python:3.11-slim, install ray[serve] xgboost shap, copy model artifact.
- docker-compose.prod.yml: all services with resource limits (backend: 512MB RAM, ml-inference: 2GB RAM), health checks, restart: unless-stopped.
- NGINX reverse proxy container: SSL termination, subdomain routing, rate limiting at proxy level.

STACK: Docker NGINX Docker Compose

DELIVERABLE All services containerized, docker-compose.prod.yml boots full stack.

8.2 Kubernetes deployment (production)

- Write K8s manifests in /infra/k8s/: Deployment, Service, Ingress, HorizontalPodAutoscaler for each service.
- Backend: min 2 replicas, max 10. ML inference: min 1, max 5. Scale on CPU > 70%.
- AWS EKS as managed Kubernetes. AWS RDS PostgreSQL (Multi-AZ) for production DB.
- AWS ElastiCache Redis cluster. Amazon MSK (managed Kafka) replacing containerized Kafka.
- cert-manager for automatic SSL via Let's Encrypt. All traffic forced to HTTPS.
- GitHub Actions CI/CD: push to main triggers tests to build to push to ECR to kubectl rollout restart.

STACK: Kubernetes AWS EKS AWS RDS ElastiCache Amazon MSK cert-manager

DELIVERABLE Production cluster on AWS with auto-scaling and zero-downtime deploys.

8.3 Subscription & payment system

- Integrate Stripe: create 3 products (Fan Free, Professional \$25/mo, Enterprise custom).
- Webhook handler at POST /stripe/webhook: checkout.session.completed upgrades tier. invoice.payment_failed downgrades to Fan.
- Build /pricing page in Next.js with Stripe Checkout redirect for Professional tier.
- Usage metering for Enterprise: count API calls per tenant per day in Redis. Bill overages via Stripe Metered Billing.
- Stripe Customer Portal link in user settings for subscription management.

STACK: Stripe Stripe Webhooks Stripe Metered Billing

DELIVERABLE Working payment flow: fan to professional upgrade via Stripe Checkout.

8.4 Security hardening & monitoring

- SQL injection prevention: all queries use parameterized statements via SQLAlchemy ORM -- never string interpolation.
- Input validation: all API inputs validated via Pydantic v2 strict mode. max_length limits on all string fields.
- Add Sentry for error tracking in FastAPI and Next.js. Alert on error rate > 1%.
- Grafana + Prometheus: instrument FastAPI with prometheus-fastapi-instrumentator. Monitor: request latency p95, Kafka consumer lag, Redis cache hit rate, ML inference latency.
- Audit logging: every data access logged to audit_log table with user_id, tenant_id, endpoint, timestamp.
- Run OWASP ZAP security scan before launch. Address all high/critical findings.

STACK: Sentry Prometheus Grafana OWASP ZAP Pydantic v2

DELIVERABLE *Production-ready: monitoring dashboards live, security scan passed.*